



University of
Applied Sciences

TECHNIKUM

WIEN

Thread synchronization (linux)

Thread synchronization

- **Complex programs require**
 - Shared data
 - Access in defined order
- **So, it requires synchronization**

Thread synchronization

- **„critical section“**
 - **Code that manipulates shared data in parallel so that it can potentially become inconsistent**
- **Ensure:**
 - while data is manipulated**
 - no other instance can access the data**
 - before the process is finalized.**

Thread synchronization

- **Cooperative process**
 - Read / write access must secure critical sections
 - Access to shared data must be synchronized (check for others, authorize)
 - Secure information exchange between threads must be guaranteed
- **Use:**
 - **Mutex (MUTual EXclusive)**
 - Special semaphore implementation
 - **Condition Variables**

Mutex - Functions

- **`pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`**
- **`int pthread_mutex_init (pthread_mutex_t *mutex,
pthread_mutexattr_t *attr);`**
- **`int pthread_mutex_destroy (pthread_mutex_t *mutex);`**

example

```
1 #include <pthread.h>
2
3  /* declare a structure with a mutex */
4  typedef struct my_struct_tag {
5      pthread_mutex_t mutex; /*protected access*/
6      int value; /*value to protect*/
7  } my_struct_t;
8
9  my_struct_t data = {PTHREAD_MUTEX_INITIALIZER, 0};
```

Lock and Unlock Mutexes

- **int pthread_mutex_lock (pthread_mutex_t *mutex);**
- **int pthread_mutex_trylock (pthread_mutex_t *mutex);**
- **int pthread_mutex_unlock (pthread_mutex_t *mutex);**

Example (1/3)

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 int a=1000; // balance of account A
5 int b=500; // balance of account B
6 int sumValue=0;
7 pthread_mutex_t mutex=PTHREAD_MUTEX_INITIALIZER;
8
9 void *move(void *arg) {
10     int status;
11     pthread_detach(pthread_self());
12
13     // critical section
14     status=pthread_mutex_lock(&mutex);
15     a=a-*((int *)arg);
16     sleep(1);
17     b=b+*((int *)arg);
18     status=pthread_mutex_unlock(&mutex);
19
20     printf("transaction completed\n");
21 }
```

Example (2/3)

```
1 void *sum(void *arg) {
2     int status;
3     pthread_detach(pthread_self());
4     sleep(1);
5
6     // critical section
7     status=pthread_mutex_lock pthread_mutex_lock(&mutex);
8     sumValue = a + b;
9     status=pthread_mutex_unlock(&mutex);
10
11     printf("Sum of accounts: %d\n", sumValue);
12 }
```

Example (3/3)

```
1 int main() {
2     pthread_t thread1, thread2;
3     int x=500;
4
5     printf("old balance: a:%d b:%d\n",a,b);
6
7     pthread_create(&thread1, NULL, move, &x);
8     pthread_create(&thread2, NULL, sum, NULL);
9
10    while (!sumValue) {sleep (1)};
11
12    printf("new balance: a:%d b:%d\n",a,b);
13
14    return 0;
15 }
```



University of
Applied Sciences

TECHNIKUM

WIEN

Threads ab C++ 11

C++ 11

- Threads added in the standard library
 - `g++ -std=c++11 -pthread`

```
1 #include <iostream>
2 #include <thread>
3
4 // This function will be called from a thread
5 void call_from_thread() {
6     std::cout << "Hello, World" << std::endl;
7 }
8
9 int main() {
10     // Launch a thread
11     std::thread t1(call_from_thread);
12
13     // Join the thread with the main thread
14     t1.join();
15
16     return 0;
17 }
```

<https://baptiste-wicht.com/posts/2012/03/cpp11-concurrency-part1-start-threads.html>

example with a group of threads

```
1 #include <iostream>
2 #include <thread>
3
4 static const int num_threads = 10;
5
6 // This function will be called from a thread
7 void call_from_thread(int tid) {
8     std::cout << "Launched by thread " << tid << std::endl;
9 }
10
11 int main() {
12     std::thread t[num_threads];
13
14     // Launch a group of threads
15     for (int i = 0; i < num_threads; ++i) {
16         t[i] = std::thread(call_from_thread, i);
17     }
18
19     std::cout << "Launched from the main\n";
20
21     // Join the threads with the main thread
22     for (int i = 0; i < num_threads; ++i) { t[i].join(); }
23     return 0;
24 }
```

mutex

```
1 #include <iostream>
2 #include <thread>
3 #include <mutex>
4
5 static const int num_threads = 10;
6 std::mutex m;
7
8 // This function will be called from a thread
9 void call_from_thread(int tid) {
10     m.lock();
11     std::cout << "Launched by thread " << tid << std::endl;
12     m.unlock();
13 }
```



University of
Applied Sciences

TECHNIKUM

WIEN

Co-Routines and Fibres

Coroutine (c++20)

- A coroutine is a function that can suspend execution to be resumed later.
- This allows for sequential code that executes asynchronously (e.g. to handle non-blocking I/O without explicit callbacks), and also supports algorithms on lazy-computed infinite sequences and other uses.
 - `co_await`, `co_yield`, `co_return`

<https://en.cppreference.com/w/cpp/language/coroutines>

co_await

- uses the `co_await` operator to suspend execution until resumed

```
task<> tcp_echo_server() {  
    char data[1024];  
    for (;;) {  
        size_t n = co_await socket.async_read_some(buffer(data));  
        co_await async_write(socket, buffer(data, n));  
    }  
}
```

<https://www.jacksondunstan.com/articles/6311>

Fiber

- **Fibers describe essentially the same concept as coroutines.**
- **The distinction is that**
 - **coroutines are a language-level construct, a form of control flow, while**
 - **fibers are a systems-level construct, viewed as threads that happen to not run in parallel.**

[[https://en.wikipedia.org/wiki/Fiber_\(computer_science\)](https://en.wikipedia.org/wiki/Fiber_(computer_science))]

- **Further reading:**
 - **[Distinguishing coroutines and fibers](#)**